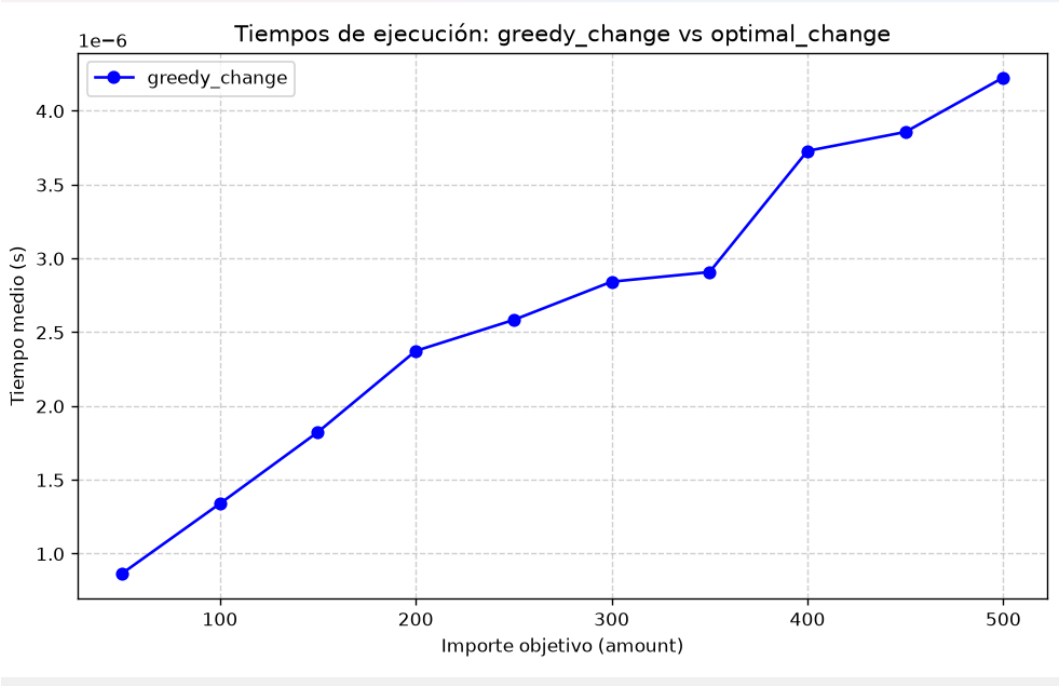
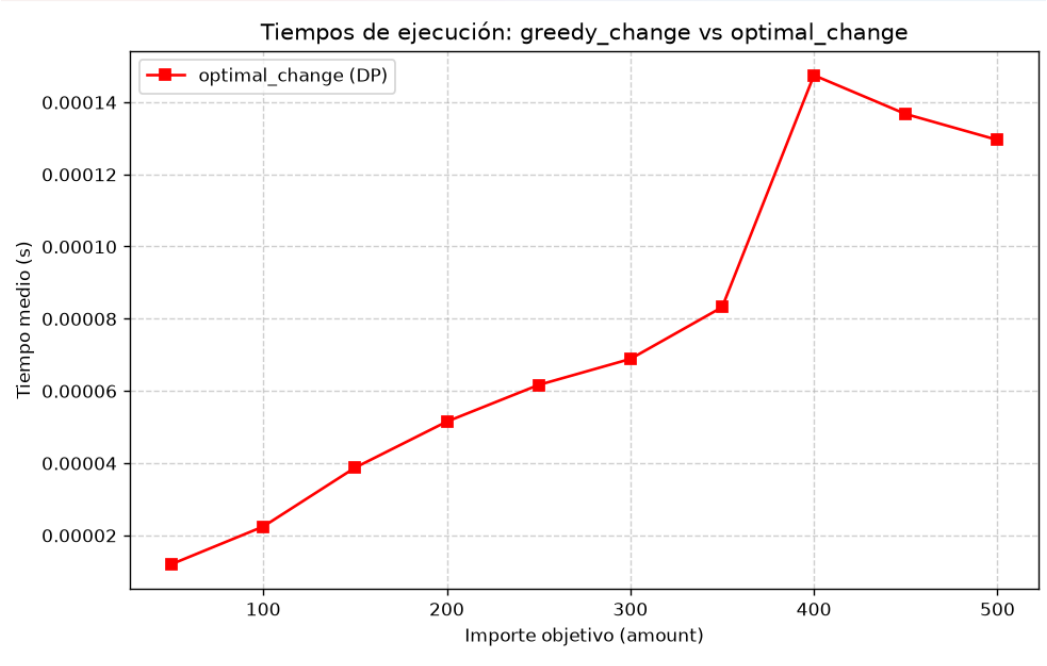
Fichero de respuestas y gráficas de Práctica de
Algoritmia 2
9-11-2026

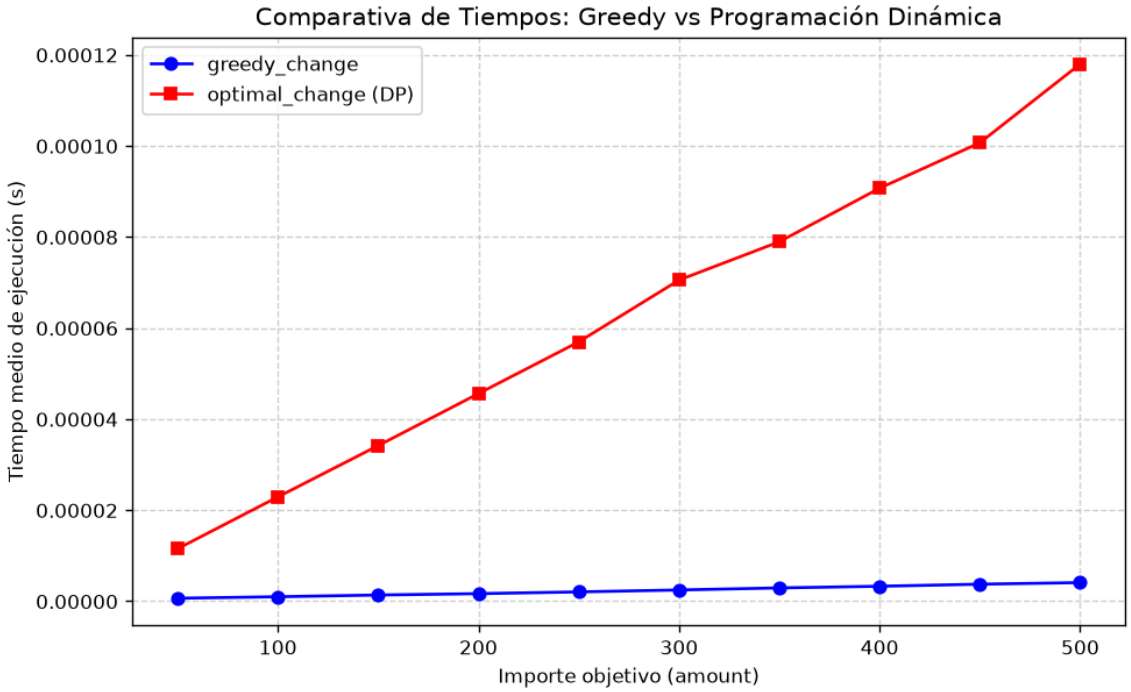
**Análisis Experimental Apartado I:**

| | Optimal | Suboptimal | Greedy_failed | Imposible |
|---|---|---|---|---|
| Sistema 1 | 500(100%) | 0(0%) | 0(0%) | 0(0%) |
| Sistema 2 | 500(100%) | (0%) | 0(0%) | 0(0%) |
| Sistema 3 | 500(100%) | (0%) | 0(0%) | 0(0%) |
| Sistema 4 | 376(75.20%9 | 124(24.80%) | 0(0%) | 0(0%) |



Fichero de respuestas y gráficas de Práctica de
Algoritmia 2
9-11-2026




Se ha tomado como ejemplo el sistema 4 para hacer la comparación de tiempos

En la estrategia codiciosa se observa que presenta un coste temporal mínimo sin embargo presenta una limitación que la calidad de los resultados no está garantizada en sistemas monetarios no canónicos, pudiendo obtener soluciones fallidas o subóptimas.

En la programación dinámica tiene una complejidad superior al tener que explorar todas las combinaciones posibles buscando la solución más óptima el tiempo de ejecución aumenta conforme crezca el importe objetivo M

**Cuestiones Apartado I:**

*1) ¿Cuál es el criterio codicioso utilizado?*

Se ha utilizado el criterio greedy que toma la moneda de mayor valor posible que o supere el importe pendiente

*2) ¿Funciona correctamente para todos los sistemas monetarios*?

No garantiza el correcto funcionamiento en sistemas monetarios no canónicos encontrando una solución subóptima o produciéndose un fallo en la estrategia dejando un sobrante imposible de cambiar

*3) ¿Cuál es la complejidad temporal de la solución codiciosa?*

Fichero de respuestas y gráficas de Práctica de
Algoritmia 2
9-11-2026

Su complejidad temporal es de O(C), donde C representa el número de monedas disponibles en el sistema monetario dado el caso del sistema que usamos ahora su complejidad sería de O(1)

*4) ¿Cuál es la complejidad temporal y espacial de la programación dinámica?*

La complejidad temporal es de O(M·N), siendo M el importe objetivo amount y N el número de monedas disponibles.

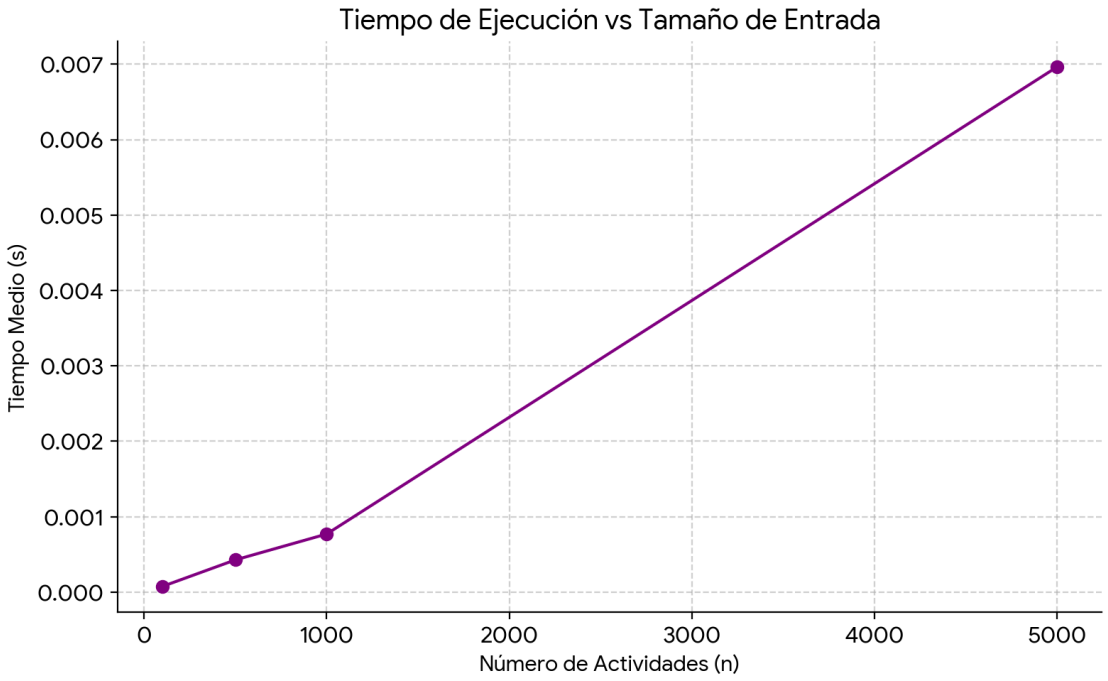
La complejidad espacial es de O(M) por la necesidad de almacenar una tabla de tamaño proporcional al importe objetivo M para guardar los resultados

*5) ¿Qué ventajas presenta la solución codiciosa, aunque no siempre garantice el óptimo?*

Su coste temporal y espacial es muy inferior que el algoritmo óptimo según la gráfica obtenida y siempre garantiza combinaciones óptimas en el sistema monetario canónico

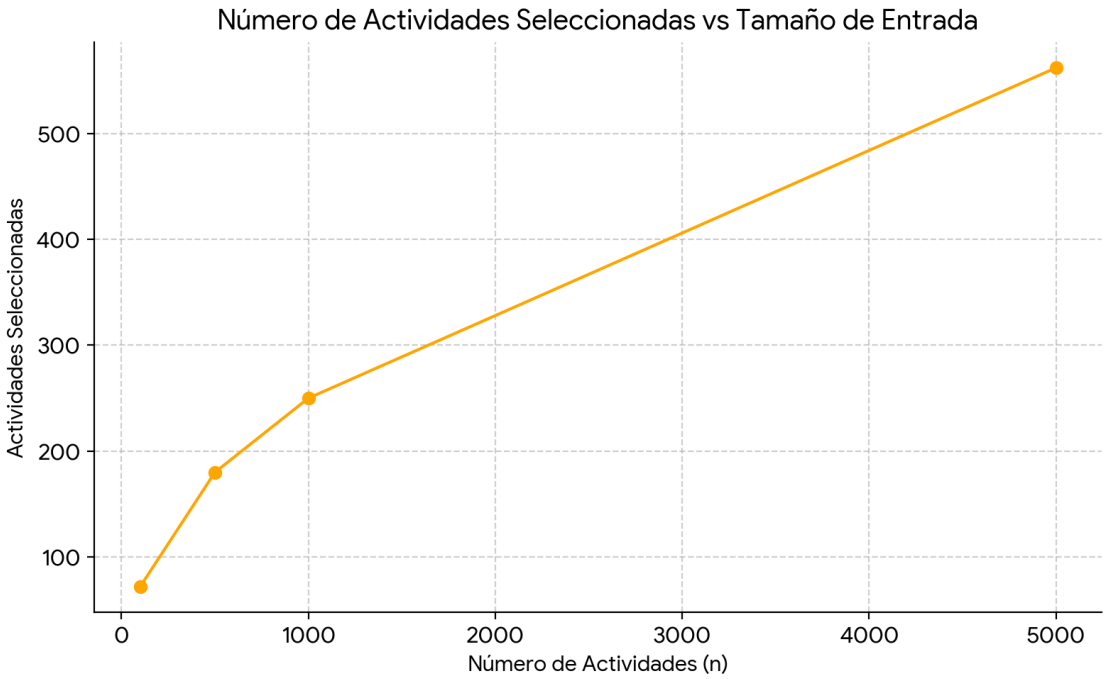
**Análisis Experimental Apartado II:**

**1.**

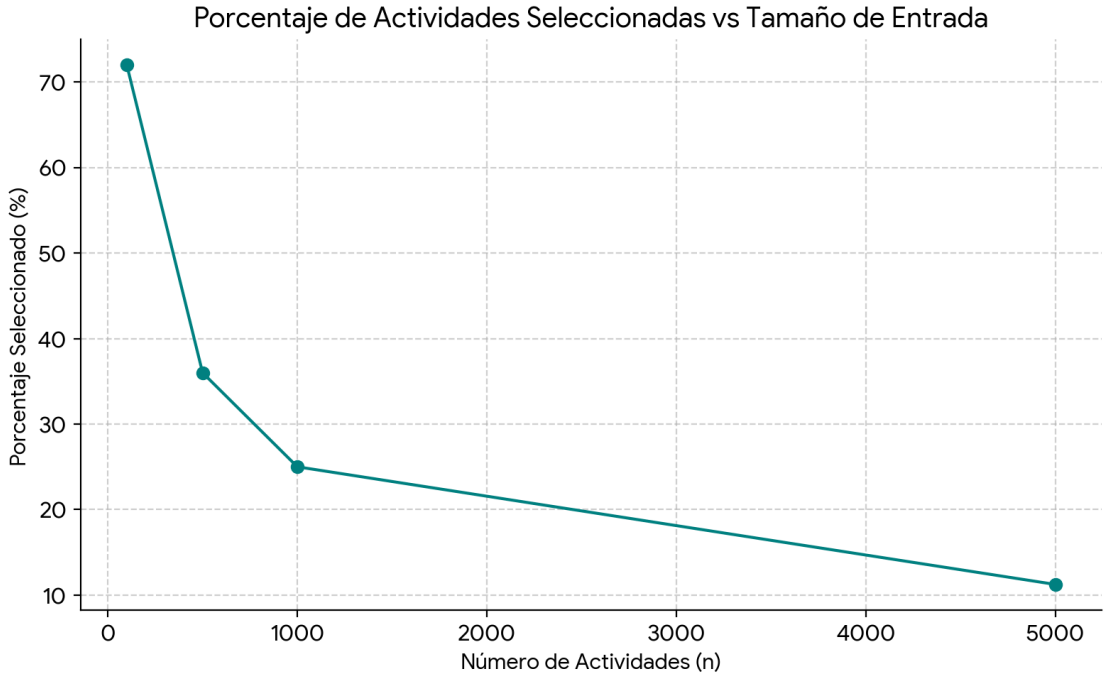


Fichero de respuestas y gráficas de Práctica de
Algoritmia 2
9-11-2026

**2.**




**3.**




**<u>Cuestiones Apartado II:</u>**

*1) ¿Cuál es el criterio codicioso utilizado?*

Seleccionar siempre la actividad compatible que termine antes, es decir, ordenarlas por tiempo de finalización ascendente.

*2)* ¿Por qué seleccionar la actividad que termina antes deja más posibilidades?

Porque libera el recurso lo más pronto posible, maximizando el tiempo libre restante en el calendario para poder encajar un mayor número de actividades posteriores.

*3) ¿Qué podría suceder si se eligiese primero la actividad que comienza antes?*

Se podría seleccionar una actividad que empiece muy pronto pero que tenga una duración larguísima, bloqueando casi todo el horario y evitando que se ejecuten muchas otras actividades compatibles.

*4) ¿Qué podría suceder si se eligiese primero la actividad de menor duración?*

Una actividad muy corta podría ubicarse justo en el tiempo de solapamiento entre dos actividades largas. Al elegir la corta, descartaríamos las otras dos, obteniendo solo 1 actividad en lugar de 2.

*5) ¿Cuál es la complejidad temporal y espacial?*

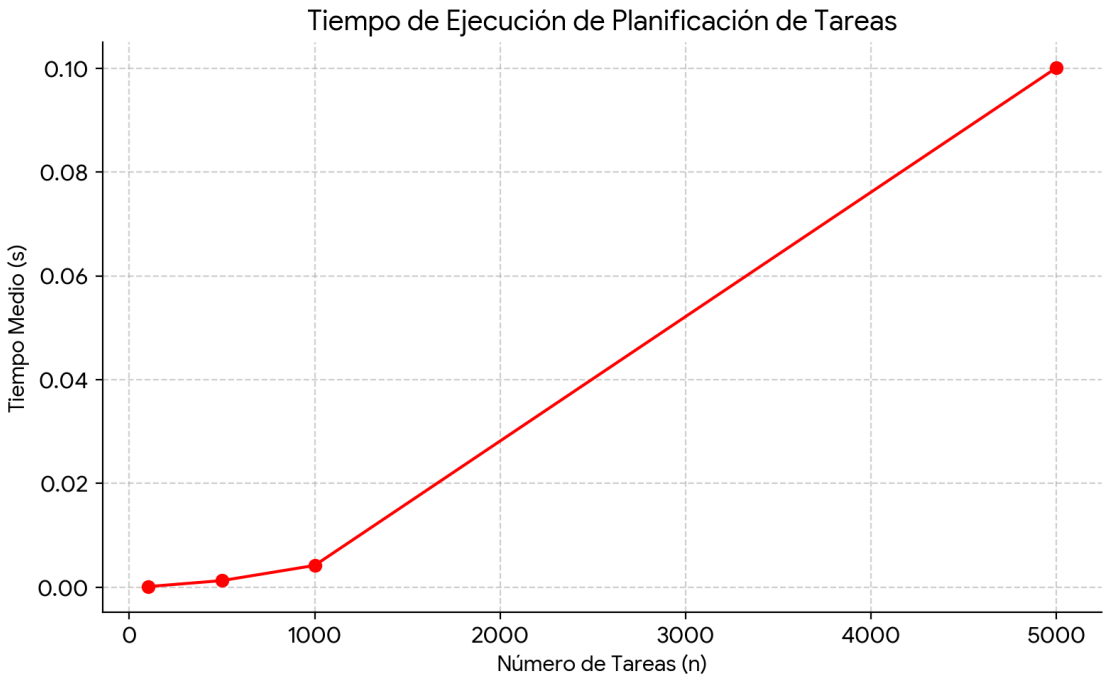
Temporal: $O(N \log N)$, dominada por el coste de ordenar la lista de actividades. El recorrido posterior para seleccionar es lineal $O(N)$.

Espacial: $O(N)$ en el peor de los casos, ya que se debe generar y devolver una nueva lista con las actividades seleccionadas.

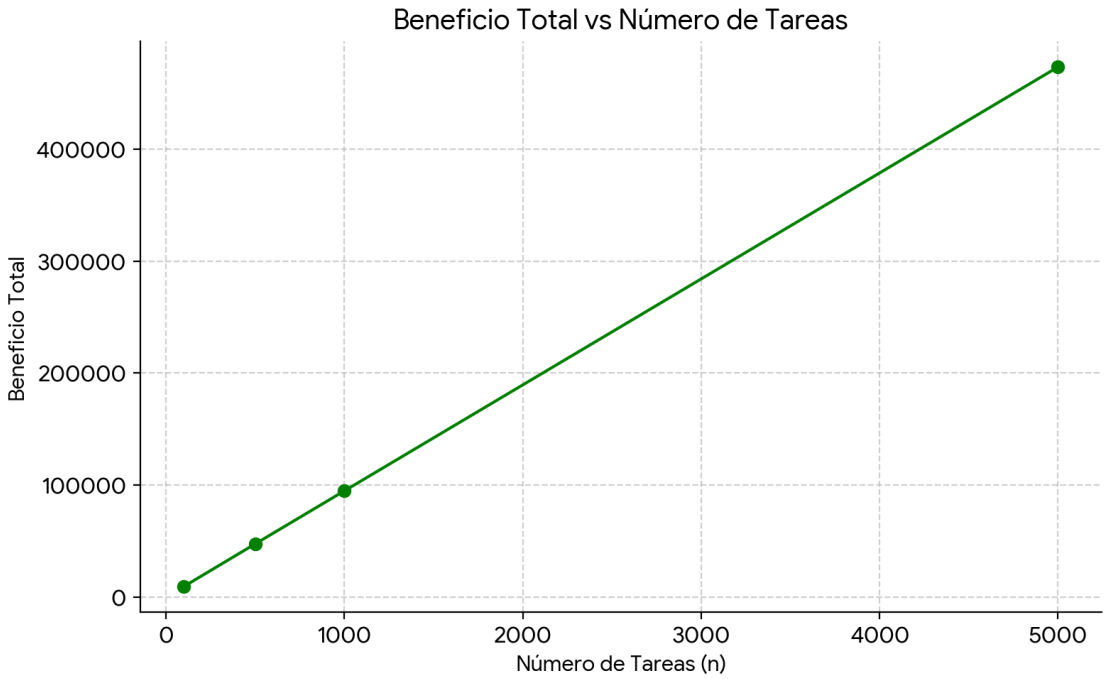
**Análisis Experimental Apartado III:**

Fichero de respuestas y gráficas de Práctica de
Algoritmia 2
9-11-2026

**1.**

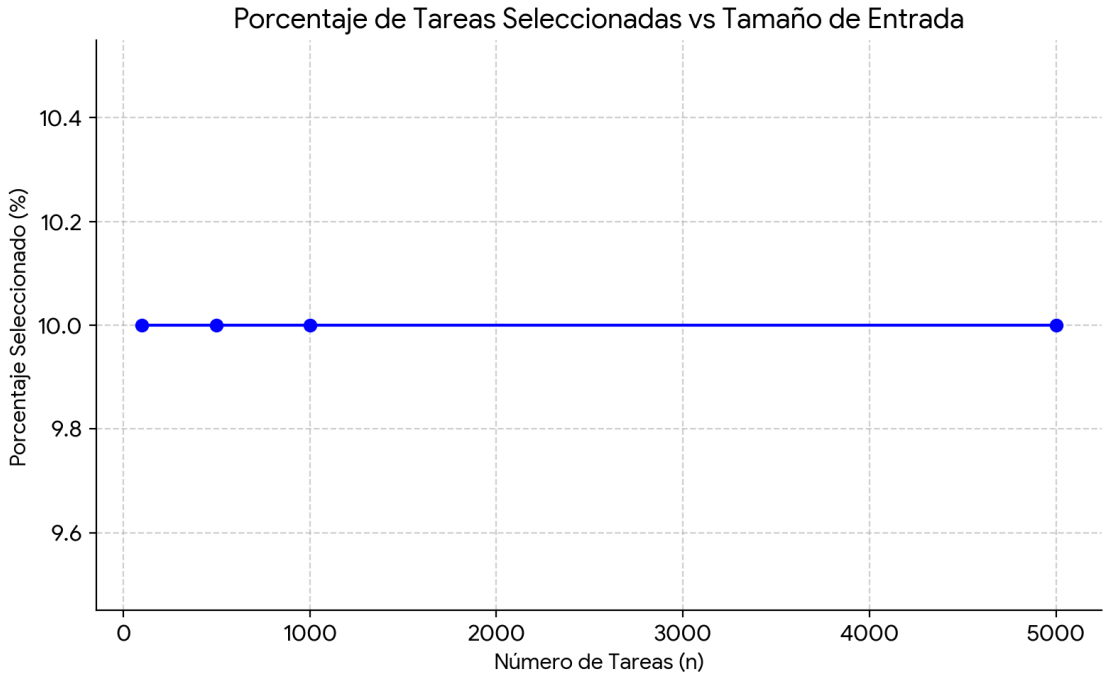



**2.**



Fichero de respuestas y gráficas de Práctica de
Algoritmia 2
9-11-2026

**3.**




**<u>Cuestiones Apartado III:</u>**

*1) ¿Cuál es el criterio codicioso utilizado?*

Priorizar las tareas de mayor beneficio y asignarlas al último intervalo libre que no supere su fecha límite.

*2) ¿Por qué se procesan primero las tareas con mayor beneficio?*

Para garantizar que los huecos del <u>calendario</u> se reserven para las tareas más valiosas, maximizando la ganancia total al descartar únicamente las menos rentables en caso de conflicto.

*3) ¿Cómo afecta la distribución de deadlines?*

Deadlines tempranos o muy concentrados provocan colisiones y un alto rechazo de tareas. Deadlines amplios y bien distribuidos reducen los solapamientos, incrementando tanto el porcentaje de selección como el beneficio.

*4) ¿Qué podría suceder si se ordenasen únicamente por deadline?*

Se priorizarían tareas urgentes independientemente de su valor económico. Esto llenaría los huecos tempranos con tareas poco rentables, bloqueando a tareas muy valiosas y reduciendo drásticamente el beneficio final.

*5) ¿Cuál es la complejidad temporal y espacial?*

Temporal: $O(N \log N + N \times D)$ al ordenar e iterar buscando huecos hacia atrás desde el deadline $D$. En esta práctica concreta ($D = N/10$), el coste se eleva a $O(N^2)$.

Espacial: $O(N + D)$ para almacenar la lista de tareas y el vector del calendario de longitud máxima $D$. Asintóticamente, equivale a $O(N)$.

**Cuestiones Apartado IV:**

*1) ¿Qué rasgos permiten afirmar que los tres algoritmos estudiados pertenecen al paradigma codicioso?*

En cada paso, los tres algoritmos seleccionan la mejor opción disponible en ese momento exacto según un criterio específico (la moneda más grande que cabe, la actividad compatible que termina antes, o la tarea con mayor beneficio).

Una vez que el algoritmo toma una decisión (seleccionar una moneda, añadir una actividad, ubicar/descartar una tarea), esta no se vuelve a evaluar ni se deshace en el futuro (no emplean *backtracking*).

Parten de una solución vacía y la construyen paso a paso incorporando los elementos que respetan las restricciones (no superar la cantidad, no solapar actividades, no superar el *deadline*).

*2) ¿Qué diferencias existen entre los tres algoritmos en complejidad temporal y espacial?*

Cambio de monedas: La complejidad temporal depende de ordenar los tipos de moneda $O(C \log C)$ y de los bucles para ir restando la cantidad (dependiente del valor de amount y las monedas). Su complejidad espacial es la menor, limitándose a $O(K)$, siendo $K$ el tamaño de la lista de monedas devuelta.

Selección de actividades: Su complejidad temporal es estrictamente $O(N \log N)$ para ordenar las $N$ actividades, seguido de un recorrido lineal $O(N)$ muy eficiente. La complejidad espacial es $O(N)$ para almacenar la nueva lista de actividades seleccionadas.

Planificación de tareas: Es el algoritmo con mayor coste. Su complejidad temporal es $O(N \log N + N \times D)$, donde $D$ es el *deadline* máximo, debido a que el bucle interior debe buscar huecos libres iterando hacia atrás. Su complejidad espacial también es mayor, $O(N + D)$, al requerir mantener en memoria una estructura para el calendario (schedule) de longitud igual al *deadline* máximo.